Examine 16-QAM Using MATLAB

This example shows how to process a binary data stream by using a communications link that consists of a baseband modulator, channel, and demodulator.

Generate Random Binary Data Stream

Define parameters.

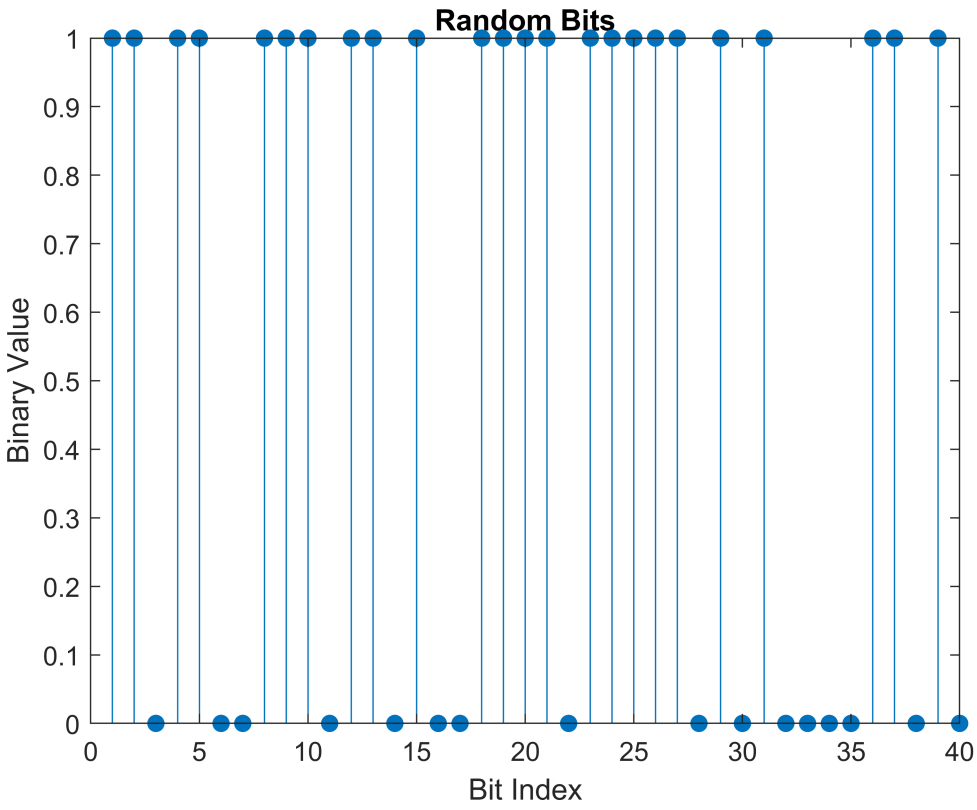
```
M = 16;      % Modulation order (alphabet size or number of points in signal constellation)
k = log2(M); % Number of bits per symbol
n = 30000;   % Number of bits to process
sps = 1;     % Number of samples per symbol (oversampling factor)
```

Set the `rng` function to its default state, or any static seed value, so that the example produces repeatable results. Then use the `randi` function to generate random binary data.

```
rng default;
dataIn = randi([0 1],n,1); % Generate vector of binary data
```

Use a stem plot to show the binary values for the first 40 bits of the random binary data stream. Use the colon (`:`) operator in the call to the `stem` function to select a portion of the binary vector.

```
stem(dataIn(1:40),'filled');
title('Random Bits');
xlabel('Bit Index');
ylabel('Binary Value');
```



Convert Binary Data to Integer-Valued Symbols

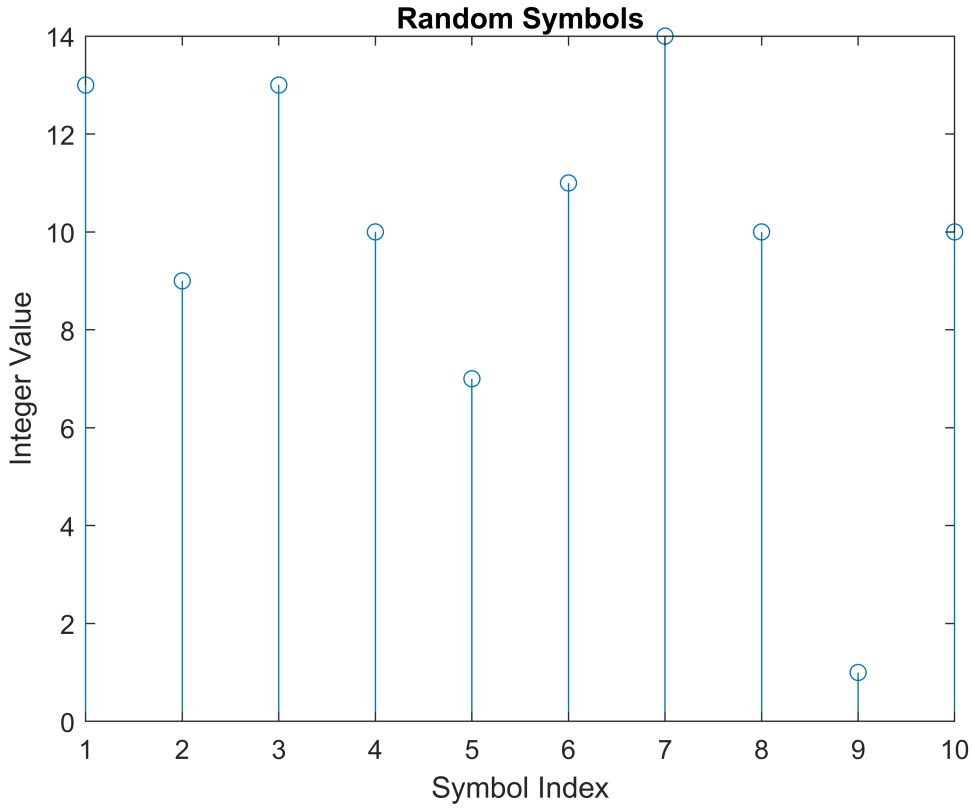
The default configuration for the `qammod` function expects integer-valued data as the input symbols to modulate. In this example, the binary data stream is preprocessed into integer values before using the `qammod` function. In particular, the `bit2int` function converts each 4-tuple to a corresponding integer in the range [0, (`M`−1)]. The modulation order, `M`, is 16 in this example.

Perform a bit-to-symbol mapping by determining the number of bits per symbol defined by $k = log_2(M)$. Then, use the `bit2int` function to convert each 4-tuple to an integer value.

```
dataSymbolsIn = bit2int(dataIn,k);
```

Plot the first 10 symbols in a stem plot.

```
figure;                     % Create new figure window.
stem(dataSymbolsIn(1:10));
title('Random Symbols');
xlabel('Symbol Index');
ylabel('Integer Value');
```



Modulate Using 16-QAM

Use the `qammod` function to apply 16-QAM modulation with phase offset of zero to the `dataSymbolsIn` column vector for natural-encoded and Gray-encoded binary bit-to-symbol mappings.

```
dataMod = qammod(dataSymbolsIn,M,'bin'); % Natural-encoded binary
dataModG = qammod(dataSymbolsIn,M);      % Gray-encoded binary
```

Add White Gaussian Noise

The modulated signal passes through the channel by using the `awgn` function with the specified signal-to-noise ratio (SNR). Convert the ratio of energy per bit to noise power spectral density (E_b/N_0) to an SNR value for use by the `awgn` function. The `sps` variable is not significant in this example but makes extending the example to use pulse shaping easier.

Calculate the SNR when the channel has an E_b/N_0 of 10 dB.

```
EbNo = 10;
snr = EbNo+10*log10(k)-10*log10(sps);
```

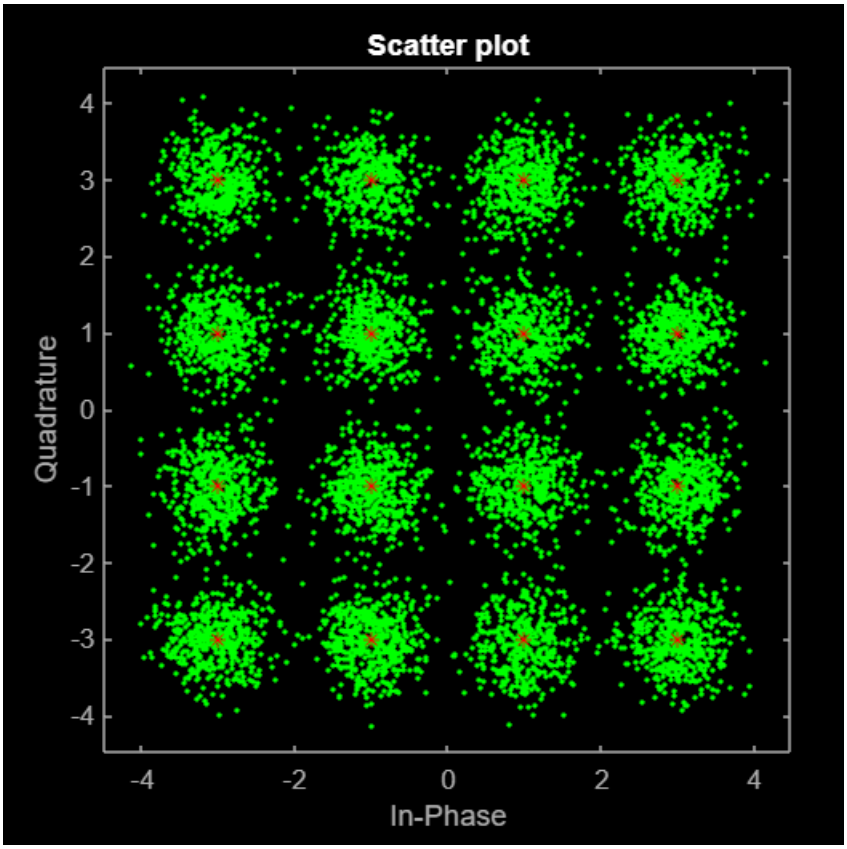
Pass the signal through the AWGN channel for the natural and Gray coded binary symbol mappings.

```
receivedSignal = awgn(dataMod,snr,'measured');
receivedSignalG = awgn(dataModG,snr,'measured');
```

Create Constellation Diagram

Use the `scatterplot` function to display the in-phase and quadrature components of the modulated signal, `dataMod`, and the noisy signal received after the channel. The effects of AWGN are present in the constellation diagram.

```
sPlotFig = scatterplot(receivedSignal,1,0,'g.');
hold on
scatterplot(dataMod,1,0,'r*',sPlotFig)
```



Demodulate 16-QAM

Use the `qamdemod` function to demodulate the received data and output integer-valued data symbols.

```
dataSymbolsOut = qamdemod(receivedSignal,M,'bin'); % Natural-encoded binary data symbols
dataSymbolsOutG = qamdemod(receivedSignalG,M);     % Gray-coded binary data symbols
```

Convert Integer-Valued Symbols to Binary Data

Use the `int2bit` function to convert the natural-encoded data symbols from the QAM demodulator into a binary vector with $(N_{\text{sym}} \times N_{\text{bits/sym}})$ length. N_{sym} is the total number of QAM symbols, and $N_{\text{bits/sym}}$ is the number of bits per symbol. For 16-QAM, $N_{\text{bits/sym}}$ = 4. Repeat the process for the Gray-encoded symbols.

Reverse the bit-to-symbol mapping performed earlier in this example.

```
dataOut = int2bit(dataSymbolsOut,k);
```

```
dataOutG = int2bit(dataSymbolsOutG,k);
```

Compute System BER

The `biterr` function calculates the bit error statistics from the original binary data stream, `dataIn`, and the received data streams, `dataOut` and `dataOutG`. Gray coding significantly reduces the BER.

Use the error rate function to compute the error statistics. Use the `fprintf` function to display the results.

```
[numErrors,ber] = biterr(dataIn,dataOut);
fprintf('\nThe binary coding bit error rate is %5.2e, based on %d errors.\n', ...
    ber,numErrors)
```

```
The binary coding bit error rate is 2.27e-03, based on 68 errors.
```

```
[numErrorsG,berG] = biterr(dataIn,dataOutG);
fprintf('\nThe Gray coding bit error rate is %5.2e, based on %d errors.\n', ...
    berG,numErrorsG)
```

```
The Gray coding bit error rate is 1.63e-03, based on 49 errors.
```

Plot Signal Constellations

Symbol Mapping for 16-QAM Constellation

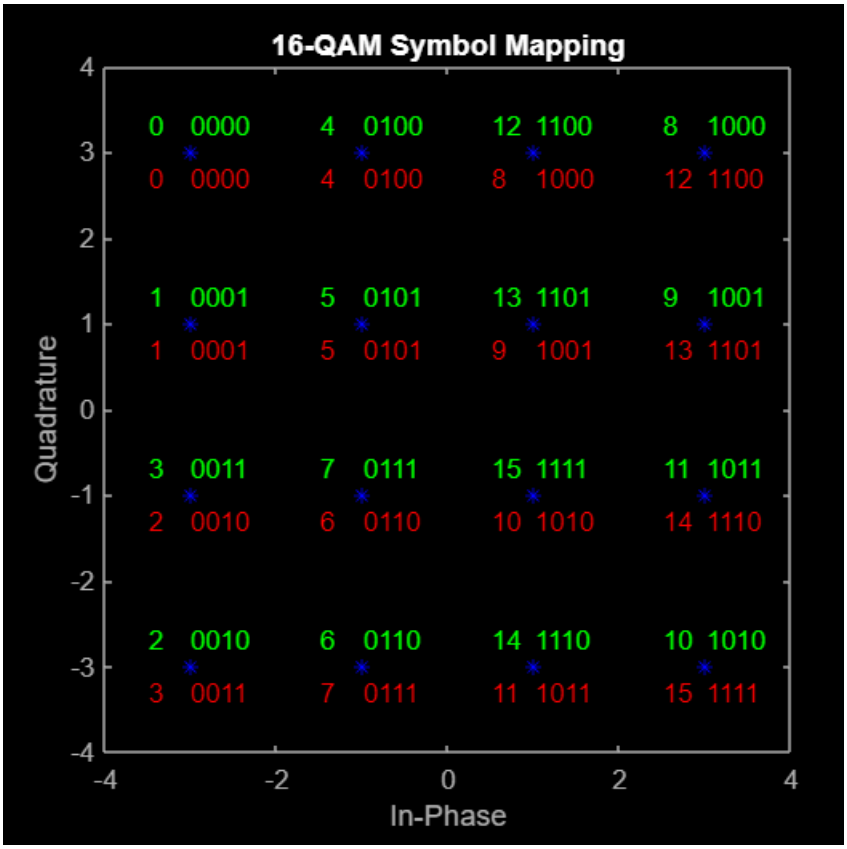
Apply 16-QAM modulation to complete sets of constellation points by using natural-coded binary symbol mapping and Gray-coded symbol mapping.

```
M = 16;                       % Modulation order
x = (0:15);                   % Integer input
symbin = qammod(x,M,'bin');   % 16-QAM output (natural-coded binary)
symgray = qammod(x,M,'gray'); % 16-QAM output (Gray-coded)
```

Use the `scatterplot` function to plot the constellation diagram and annotate it with natural (red) and Gray (black) binary representations of the constellation points.

```
scatterplot(symgray,1,0,'b*');
for k = 1:M
    text(real(symgray(k)) - 0.0,imag(symgray(k)) + 0.3,...
        dec2base(x(k),2,4),'Color',[0 1 0]);
     text(real(symgray(k)) - 0.5,imag(symgray(k)) + 0.3,...
         num2str(x(k)),'Color',[0 1 0]);

    text(real(symbin(k)) - 0.0,imag(symbin(k)) - 0.3,...
        dec2base(x(k),2,4),'Color',[1 0 0]);
    text(real(symbin(k)) - 0.5,imag(symbin(k)) - 0.3,...
        num2str(x(k)),'Color',[1 0 0]);
end
title('16-QAM Symbol Mapping')
axis([-4 4 -4 4])
```



Copyright 2019-2021 The MathWorks, Inc.